Incident Audit: Stale Status Claims, and a Self-Inflicted Cost Regression

SQL_TEST_01 — Ask-Your-Database Agent (Build 6/7) — 2026-08-10

Scope: a routine verification pass, three days after the 2026-08-07 incident, that found two confidently-wrong status claims in live answers — then, while fixing them, introduced a real cost regression of its own, caught and fixed before being called done rather than after.

Timezone: Pacific (UTC-7), converted from the container's UTC-recorded logs.

Evidence basis: every timestamp and token count below is pulled directly from `logs/tool_calls.jsonl`, `agent_scratch.chat_rounds`, direct `docker exec` / `psql` queries against `docs.chunks`, and `git log` — not reconstructed from memory.

Executive summary

Two live browser questions — "list all open and deferred items" and "is there a scope for full deployment of Streamlit app?" — both came back green-badged and `supported: true`, and both were wrong about current status. The first called the 2026-08-07 deployment-verification gap "still unresolved"; the second repeated the claim as an active "known deployment challenge." Direct verification found the opposite: the gap was closed same-day on 2026-08-07, and the running container's code matched git HEAD exactly, byte for byte. A second, smaller claim — that `docs.chunks` still had orphaned rows from before the 2026-08-05 reorg — was also stale; a direct query confirmed zero such rows exist. Root cause for the first: `MAX_FIELD_CHARS` truncates every retrieved field at 500 characters, and the cheat sheet's incident entries are written problem-first, resolution-second — so the truncation cutoff landed, with almost comedic precision, right before the word "Fixed" every time that entry was retrieved. The verifier never caught it because the claim was genuinely consistent with the incomplete evidence it was given — the same `supported: true` blind spot from the 2026-08-07 audit, in a fourth shape: a stale *temporal* inference instead of a fabricated fact. The first fix attempt — a caution rule telling the model not to conclude "unresolved" from a truncated chunk — corrected both answers, but at a real cost: one question's total went from a few thousand tokens to **202,821**, because the rule gave the model permission to keep searching rather than a bounded, cheap action. That is the exact "soft exploratory instruction causes more searching, not less" anti-pattern this project's own 2026-08-06 incident already named and moved away from — reintroduced here by accident, caught by re-testing before being called done, and fixed the same way that original incident was: by resolving the ambiguity once, as data (a consolidated, kept-current "Known Open & Deferred Items" table), not as a heuristic the model has to re-derive and re-search for on every question. Called out directly afterward: that table fixed the two questions tested, not the mechanism that produced the original bug. A direct check found **127 of ~532 chunks** already exceeded the truncation cap (up to 6,118 characters) - the deployment-gap entry was one instance of a general problem in `extract_doc_chunks.py`'s chunking strategy, which had no size discipline at all. The actual fix moved chunk-splitting to extraction time: any chunk over a safe target now gets broken up on sentence (or, failing that, word) boundaries before it's ever stored, so no chunk this pipeline produces can be long enough to have anything truncated out of it, for any question, about any topic - re-verified across the real, fully re-extracted dataset (769 chunks, zero over the cap, down from 127).

Timeline

Phase 1 — Two live answers, both confidently wrong about current status

`18:21 – 18:29 PDT`

`18:21`	"List all open and deferred items for this project" — 4 tool calls, 41,417 tokens	`5425deb8fa1e`
`18:28`	"Is there a scope for full deployment of Streamlit app?" — 3 tool calls, 29,377 tokens	`72eafb19faad`

Both answers were internally coherent, cited real evidence, and were marked `supported: true` by the verifier. Both stated the 2026-08-07 deployment-verification gap as a current, unresolved problem — one calling it "still unresolved," the other "one unresolved scope gap." A third claim (orphaned `docs.chunks` rows from the pre-reorg era, "not yet done") also appeared, sourced from a 2026-08-05 handoff brief that had never been updated.

> **Direct verification, not assumed:** `docker exec askdb_streamlit grep` confirmed the running container's `agent.py` matches the current git HEAD byte for byte. `SELECT COUNT(*) FROM docs.chunks WHERE source_file LIKE 'PROJECT_DIAGRAMS%'` returned zero rows. Both claims were stale, not current findings.

Phase 2 — Root cause: a truncation cutoff that lands right before the word "Fixed"

The retrieved chunk behind the deployment-gap claim (`chunk_id 6988`) is 3,303 characters long. `MAX_FIELD_CHARS = 500` caps every field a query returns, so the model only ever saw the first ~500 characters — which happened to end mid-sentence, right before "`Fixed, then asked directly whether the underlying docs.chunks refresh step could be made automatic...`". The cheat sheet's own writing convention (problem stated first, resolution stated second, within the same entry) meant this wasn't a one-off: any sufficiently long incident entry has the same structural risk. The verifier's `supported: true` verdict was not wrong on its own terms — the claim really was consistent with the truncated evidence retrieved. It just wasn't consistent with the complete entry.

> **The recurring shape:** this is the fourth confirmed live instance of the same underlying blind spot from the 2026-08-07 audit — `supported: true` proves consistency with the evidence gathered, never that the right (or complete) evidence was gathered. Incomplete `search_docs` sample (Phase 5), frozen snapshot over living file (Phase 8b), fabricated classification on `run_sql_query` (Phase 11) — and now, a genuinely retrieved, genuinely relevant chunk that got cut off exactly where the answer to the question being asked began.

Phase 3 — First fix: correctness restored, cost exploded

Two changes went out together: the deployment-gap cheat sheet entry got a front-loaded **"(RESOLVED..."** marker within its first 90 characters (safely inside any truncation window), and `SYSTEM_PROMPT` gained a rule telling the model not to conclude "unresolved" from a truncated chunk alone — with a second clause suggesting it could *"query nearby chunk_ids in the same source_file for a follow-up that confirms current status either way."* Re-tested live on a fresh rebuild: both answers became correct. Both also became enormous.

QUESTION	TOKENS	TOOL CALLS
"List all open and deferred items" (re-test)	202,821	13
"Is there a scope for full deployment...?" (re-test)	117,146	7

Self-inflicted regression, caught before being called done: the "or query nearby chunk_ids" clause gave the model a soft, open-ended license to keep searching instead of a bounded action. Combined with a genuinely hard retrieval problem (the cheat sheet has no single section answering "what's open," so the model tried a dozen different ILIKE keyword guesses - `%open%`, `%TODO%`, `%pending%`, `%outstanding%`, `%backlog%` - most returning nothing), this produced the exact same shape of blowup this project's own 2026-08-06 incident already diagnosed and moved away from: *"a soft 'pull the full list when ambiguous' option caused more exploration, not less. More expensive than the wrong answer it replaced."* Re-introduced here, three days later, by a different rule solving a different problem the same wrong way.

Phase 4 — Second fix: resolve it as data, the way the pattern actually gets closed

Consistent with the throughline already established across every prior incident in this project - resolve an ambiguity once, as a stated fact, not as a heuristic the model re-derives from scratch on every question - the real fix was a new, consolidated **"Known Open & Deferred Items"** table added directly to the cheat sheet: five rows, one per item, each starting with an explicit status word (`RESOLVED` , `OPEN, DEFERRED BY DESIGN` , or `CLOSED, HISTORICAL DECISION`), kept current as of this audit. `SYSTEM_PROMPT` 's existing SQL-routing rule (the same one that already routes "every file," "every command," "every migration," and "latest X" to a direct query) was extended to name "open/deferred/outstanding items" as another tracked category, pointing directly at this table. The exploratory "query nearby chunk_ids" clause was removed entirely and replaced with a flat instruction: hedge once, don't chase the tail with more queries - the table exists specifically so this class of question never needs that in the first place.

> **Verified live, on a freshly rebuilt and recreated container:** both questions re-run from scratch.

QUESTION	TOKENS	TOOL CALLS	CORRECTNESS
"List all open and deferred items"	17,872	3	Correctly lists exactly the 2 genuinely open items, correctly notes the 2 closed ones
"Is there a scope for full deployment...?"	23,954	3	Correctly states the Build 6 goal was achieved, deployment gap explicitly cited as resolved

Both verified `supported: true` , with the verifier's reasoning citing the specific status-labeled chunks directly. Full test suite: 39/39 (this was entirely a documentation/prompt-wording fix; no code path changed, so no new unit test was needed - validated behaviorally against the live API instead, the only real way to check retrieval and wording together).

Phase 5 — Called out directly: Phase 4 fixed this situation, not the logic behind it

Phase 4's table fixed the two questions actually tested. It didn't touch the mechanism that produced the original bug: `extract_callout_chunks()`, `extract_finding_chunks()`, and `extract_table_row_chunks()` take an entire callout div, finding div, or table cell as *one* chunk, with no size limit at all. Checked directly: **127 of ~532 chunks** already exceeded `MAX_FIELD_CHARS` (500) before this fix, up to **6,118 characters** in one case. The deployment-gap entry was one instance of a general problem, not a special case - any one of the other 126 could produce the identical failure on a completely different question, about a completely different topic, with no table anywhere to route it to.

> **Root cause, the actual one this time:** the chunking strategy has no size discipline. A callout can run to any length the prose happens to reach, and nothing downstream ever revisits that. Patching one long entry's wording, or building one consolidated table for one question shape, both treat a symptom - the generator of arbitrarily long chunks was still live and would produce the next one on the next edit.

Fix: chunk-splitting moved to extraction time, not query time. A new `_split_long_text()` in `extract_doc_chunks.py` breaks any chunk over `CHUNK_SPLIT_TARGET_CHARS` (450 - a safety margin under the live 500-char cap) on sentence boundaries first, falling back to word-boundary wrapping for the rare single sentence that's itself oversized (this project's prose leans on em-dashes over periods, so a period-only split isn't sufficient on its own). Applied once, centrally, after all four extraction functions run, so the guarantee covers every chunk this pipeline produces - not just whichever ones a specific incident happened to surface. Re-ran the real extraction across all 13 source files: chunk count grew from 532 to 769 (more, smaller pieces, same total content), and a direct query confirms **zero chunks exceed 500 characters, dataset-wide** - down from 127.

While in the file: `extract_doc_chunks.py` ran its entire connect-and-extract body unconditionally at module import time, with no `if __name__ == "__main__":` guard - meaning importing `_split_long_text` for a unit test would have opened a live database connection and re-extracted every doc as a side effect of the import alone. Wrapped in a `main()` function, guarded the normal way; the script's own invocation (`python src/extract_doc_chunks.py`) is unaffected.

> **Verified live, on a freshly rebuilt and recreated container, after the real extraction re-run:** both Phase 4 questions re-tested, plus a third, unrelated question to confirm chunk-order-dependent recency routing (`ORDER BY chunk_id DESC`) still holds correctly with far more numerous, smaller chunks.

QUESTION	TOKENS	CORRECTNESS

"List all open and deferred items"	15,491	Correct, `supported: true`
"Is there a scope for full deployment...?"	18,992	Correct, `supported: true`
"What are the latest commits?" (regression check)	17,396	Correctly ordered newest-first (2026-08-10 → 08-07 → 08-06 → 08-05); flagged `supported: false` for an unrelated, pre-existing terminology point (the cheat sheet is a documentation summary, not a literal git log) - a legitimate verifier catch, not caused by this fix, out of scope here

5 new tests in `tests/test_extract_doc_chunks.py`, covering the pure `_split_long_text()` function directly: short text passes through unchanged, long text splits on sentence boundaries with no content lost, every split piece stays under the live 500-char cap (checked against that real number, not just the internal target), a single oversized sentence still gets hard-wrapped, and word-level content is fully preserved end to end. Full suite: 44/44.

Before / after

QUESTION	STALE-CLAIM RUN	REGRESSION RUN	SITUATION FIX (PHASE 4)	LOGIC FIX (PHASE 5)	CORRECTNESS
"List all open and deferred items"	41,417	202,821	17,872	15,491	Wrong → correct but 4.9x costlier → correct and cheap → correct and cheapest
"Is there a scope for full deployment...?"	29,377	117,146	23,954	18,992	Wrong → correct but 4x costlier → correct and cheap → correct and cheapest

The regression column is a real, deployed, re-tested state - not a hypothetical worst case. It was live for the length of this investigation before being caught and replaced. Phase 4's fix was real and correct for both questions actually tested, but only Phase 5 fixed the mechanism - the 126 other chunks that were also over the truncation cap before this audit are the reason Phase 4 alone wasn't the end of it.

Decision log

DECISION	WHY
Verify the agent's own self-report against direct evidence rather than trust it	The project's own standing practice - a green badge and a `supported: true` verdict prove internal consistency, never completeness or currency. Direct `docker exec` / `psql` checks are what actually caught both stale claims.
Re-test the first fix before calling it done, not just trust that correctness improved	The exact discipline that caught the 202,821-token regression. A fix that only gets checked for correctness, not cost, can pass review and still be strictly worse than what it replaced.
Fix the retrieval gap as consolidated data, not a smarter prompt rule alone	A prompt rule can bound behavior, but it can't manufacture an efficient index that doesn't exist. The cheat sheet had no single place answering "what's open" - the model's expensive keyword-guessing was a symptom of that gap, not just of the caution rule.
Remove the exploratory "query nearby chunk_ids" clause entirely rather than tune it	Any soft permission to keep searching re-opens the same cost risk for the next question this pattern applies to. A flat "hedge once, don't chase" instruction plus a real data fix closes it structurally instead of by degree.

Lessons

- A truncation cutoff can land exactly where the answer to the question begins - not hypothetically, confirmed twice in the same chunk pattern (problem-first, resolution-second prose, cut at a fixed character count). When a project's own writing convention is consistent, a fixed-size truncation risk isn't random - it's structurally biased toward hiding the same kind of information every time.
- A caution rule that tells a model "don't conclude X, verify further" is not automatically safe just because it improves correctness. Without a bounded, cheap verification path

already available, "verify further" becomes "search until satisfied" - and satisfaction has no natural cost ceiling.

- This project already named this exact anti-pattern once, in the 2026-08-06 incident, and it recurred anyway three days later in an unrelated fix. A lesson learned from one incident doesn't automatically transfer to the next one that has the same shape - it has to be checked for, not just remembered.
- Re-testing after a fix has to check cost, not just correctness. This session's own workflow - rebuild, redeploy, re-test live, read the actual token usage - is what caught the regression before it shipped as "done." A fix that's only checked against the original failure mode can introduce a new one invisibly.
- `supported: true` continues to only mean "consistent with the evidence gathered." A fourth confirmed live shape of the same blind spot: a real, relevant, correctly-retrieved chunk that was simply cut off before it said the thing that mattered.
- A fix that resolves the specific case in front of you isn't the same as a fix that resolves the mechanism behind it - and the difference is checkable, not just philosophical. "127 of ~532 chunks already exceed the cap" was one query away the whole time; the first fix shipped without anyone running it.